

FYP1-MidReport-F24-167-D-Intelli-Datum

Project Team

Zinoor Fatima 21I-1368
Ume Khadija 21I-0427
Haniya Usman 21I-0748

Session 2021-2025

Supervised by

Dr. Muhammad Asif Naeem

Co-Supervised by

Mr. Muhammad Aamir Gulzar



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

June, 2025

Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Scope	2
1.3	Modules	3
1.3.1	File Input Module	3
1.3.2	Delimiter Separated File Handler Module	3
1.3.3	Fixed Length File Handler Module	4
1.3.4	XML File Parser Module	4
1.3.5	Line Data Handler Module	4
1.3.6	WebApp	5
1.3.7	Data Mapping	5
1.4	User Classes and Characteristics	6
2	Project Requirements	7
2.1	Use-case/Event Response Table	7
2.1.1	Use-case Diagram and use-case package diagram	7
2.1.1.1	Use-case Diagram	7
2.1.1.2	Use-case Package Diagram	8
2.1.1.3	Detailed Use-cases	9
2.1.2	Event-Response Table	12
2.1.2.1	Manage account	12
2.1.2.2	Manage documents	13
2.1.2.3	Processing files	13
2.1.2.4	View output	14
2.1.2.5	Communicate system	14
2.2	Functional Requirements	15
2.2.1	File Input Module	15
2.2.2	Delimiter Separated File Handler Module	16
2.2.3	Fixed Length File Handler Module	16
2.2.4	XML File Parser Module	16
2.2.5	Line Data Handler Module	16
2.2.6	WebApp	17

2.2.7	Data Mapping	17
2.3	Non-Functional Requirements	17
2.3.1	Reliability	17
2.3.2	Usability	18
2.3.3	Performance	18
2.3.4	Maintainability	18
2.4	Domain Model	18
3	System Overview	21
3.1	Architectural Design	21
3.1.1	Client	21
3.1.2	Cloud Service	21
3.1.3	Database	22
3.1.4	Data Mapping	22
3.2	Design Models	22
3.2.1	Activity Diagram	22
3.2.1.1	Client Interface	22
3.2.1.2	Processing File	23
3.2.1.3	Communication Processing	23
3.2.1.4	Document Management	23
3.2.2	Class Diagram	24
3.2.2.1	User Classes	24
3.2.2.2	File Classes	24
3.2.2.3	Dictionary Classes	25
3.2.3	System Sequence Diagram	27
3.2.4	Sequence Diagram	28
3.2.4.1	Processing File	28
3.2.4.2	Manage Accounts	28
3.2.4.3	View Output	29
3.2.4.4	Manage Documents	30
3.2.4.5	Communicate with User	32
3.2.5	State Transition Diagram	32
3.2.6	Deployment Diagram	33
3.2.7	Data-Flow Diagram	34
3.3	Data Design	35
3.3.1	File Processing Overview	36
3.3.2	Data Mapping	37
3.3.3	Data Storage	37
4	Conclusion and Future Work	39

References	41
-------------------	-----------

List of Figures

2.1	Use-case Diagram	8
2.2	Use-case Package Diagram	9
2.3	Domain Model Diagram	19
3.1	Architecture Diagram	22
3.2	Activity Diagram	24
3.3	Class Diagram	26
3.4	System Sequence Diagram	27
3.5	File Processing Sequence Diagram	28
3.6	Manage Account Sequence Diagram	29
3.7	Output View Sequence diagram	30
3.8	Manage Documents Sequence Diagrams	31
3.9	Communicate with User Sequence Diagram	32
3.10	State Transition Diagram	33
3.11	Deployment Diagram	34
3.12	Dataflow Diagram	35
3.13	Example Dictionary	36
3.14	Entity Relationship Diagram	37

List of Tables

1.1	User Classes and Characteristics	6
2.1	Manage Account Use Case	10
2.2	Manage Documents Use Case	10
2.4	View Output Use Case	11
2.5	Communicate System Use Case	11
2.3	Processing Files Use Case	12

Chapter 1

Introduction

IntelliDatum is a smart data management web app that uses AI to automate the processing and structuring of unstructured data files into structured human understandable dictionaries. This project is in collaboration with Elixir Technologies, a technology and services company. The aim is to solve the challenge of manual intervention required for structuring and mapping data to dictionaries.

This report describes in detail the requirements for the IntelliDatum. The document gives an overview of the scope of the project, lists down the functional and nonfunctional requirements, and the architecture of the web app IntelliDatum. This document is intended to be used by numerous types of users. Developers can use the document to understand the requirements and the architectural designs to code accordingly. Testers can use it to design test cases according to the requirements. It can be used by project managers to set milestones and make sure all requirements are delivered. It can be used by the stakeholders to make sure that the agreed requirements are met and delivered. Furthermore, the document is intended for supervisors and FAST University, so it is clear what the students are developing in their Final Year Project. The final product can later be evaluated in accordance with the requirements stated in this document.

1.1 Problem Statement

Data is a crucial part of businesses all around the world. Day by day, with the growing amount of information, we have a large amount of incoming data. It is predicted that the volume of global data will rise to 175 Zettabytes by 2025. This brings about the challenge of data management in an accurate and timely manner.[1] Companies like Elixir Technologies receive varying volumes of customer data daily. This data is often in the form of unstructured files. Unstructured files are those files that are not presented in the form of a preset model or schema.[2] The first step in handling this data is to develop a

data dictionary that provides a blueprint for further data management. Data management is the practice of collecting, organizing, protecting, and storing an organization's data so it can be analyzed for business decisions.[3]

Currently in Elixir, this process of dictionary extraction from files is handled manually by a data designer, who processes these files by identifying missing columns, interpreting the data, and generating a suitable dictionary for it. The data received generally falls into four main types—delimiter-separated files, fixed-length files, line data files, and XML files. The main challenge is that these files often contain missing columns or lack a uniform structure, requiring manual intervention by data designers to parse through and map the data correctly. Each file type has unique requirements, which complicate the process of dictionary extraction, slow down data analysis and data mapping, and hinder automation.

By leveraging machine learning and generative AI, this entire process can be automated. There is a need to create algorithms that can detect missing columns, identify patterns, and dynamically interpret the schema, eliminating the need for manual labor. This will streamline the data and ensure that there is consistency and integrity across the entire system. It will also give way to automation of customized documents like certificates or reports by creating templates that map data from files using structured dictionaries.

1.2 Scope

The scope of this project is to develop a user-friendly Web App, IntelliDatum, which is integrated with a self-developed cloud-based backend service. The service is developed to extract structured information from heterogeneous data files. IntelliDatum will serve as an interface for users so they can upload numerous files of varying file formats and types, including delimiter-separated files, fixed-length files, XML files, and unstructured line data files. The service will handle diverse data types such as strings, dates, times, numeric values, and booleans.

A core component of IntelliDatum is an AI model that learns to recognize patterns, infer field boundaries, and group records across different file structures by analyzing the context of the data within each file. The model will interpret and convert it into structured dictionaries, making the data uniform across formats. The model will be deployed as a scalable cloud service. The application will feature advanced functionalities such as schema validation, error detection, and the ability to process files containing single records, multiple records, single documents, or multiple documents simultaneously.

Another feature of IntelliDatum is a communication and data mapping module. This module will utilize the generated dictionaries to map data from input files to predefined document templates, allowing users to easily create customized documents such as certificates, reports, and much more. Instead of building or fine-tuning an open-source LLM

(Large Language Model) specifically for this purpose, IntelliDatum will utilize an existing LLM or an API for accurate and efficient results.

The platform will allow users to manage, view, and update generated dictionaries, as well as create documents through templates. All in all, IntelliDatum will allow for easy file submission, dictionary generation, and data mapping, providing an accessible, efficient, and user-friendly experience for managing and structuring unstructured data files.

1.3 Modules

1.3.1 File Input Module

This module is responsible for receiving different types of data files as input and analyzing their structure. It includes file type detection, extracting file metadata (single vs multi document, single vs multi record etc), and matching new data files against predefined dictionaries to check if they fit.

1. File Type Detection
2. File Metadata Extraction
3. Dictionary Matching

1.3.2 Delimiter Separated File Handler Module

This module handles files with delimiter-separated values. It detects delimiters, extracts headers (if there are any), and generates dictionaries for both single and multiple-record files.

1. Delimiter Detection
2. Header Detection
3. Single Record Handling
4. Multiple Record Handling
5. Single and Multi Document Handling
6. Dictionary Generation

1.3.3 Fixed Length File Handler Module

This module processes fixed-length files where each field has a fixed length. It supports handling single or multiple records and generates dictionaries from these files.

1. Record Length Identification
2. Header Detection
3. Single Record Handling
4. Multiple Record Handling
5. Single and Multi Document Handling
6. Dictionary Generation

1.3.4 XML File Parser Module

This module is responsible for parsing XML files, extracting data from the structured format, and generating dictionaries that represent the hierarchical data structure.

1. File Parsing
2. Dictionary Generation

1.3.5 Line Data Handler Module

This module handles line data files that may lack a consistent structure. It recognizes patterns using positions and context to create dictionaries from the unstructured content.

1. Pattern Recognition using
 - (a) Position
 - (b) Context
2. Heuristic-based Field Mapping
3. Single and Multi Document File Handling
4. Dictionary Generation

1.3.6 WebApp

This module is a client web app that serves as the interface for clients to interact with IntelliDatum. It provides functionalities such as uploading files, viewing generated outputs, and managing documents.

1. UI/UX
2. Frontend Development
3. Backend Development
4. Integration
5. Testing
6. Deployment

1.3.7 Data Mapping

This module supports mapping extracted data to predefined templates. It allows users to generate documents, such as certificates, based on customer data and defined templates.

1. Message Interpretation
2. Create and Manage Templates
3. Data Mapping to Templates
4. Automated Document Generation

1.4 User Classes and Characteristics

Table 1.1: User Classes and Characteristics

User class	Description
Enterprise Users	Enterprise users are organizations with large amounts of incoming unstructured data (for example from banks). The organizations that have multiple clients sending data in different formats can utilize IntelliDatum for fast dictionary generation to structure the data and then make use of it.
IT Administrators	IT Administrators are users who oversee systems administration and user management. They possess strong technical and problem-solving skills and provide user support. They can use IntelliDatum when handling large amounts of user data to structure and convert it into templates, for example, reports, certificates, and much more.
Data Managers	Data Managers are users that manage and organize data. These users have knowledge about data and aim to organize it in a format that can be usable. They can use IntelliDatum to structure their data.
End Users (Clients)	End users are the users who want to try converting data into specific formats. Users with low technical skills aim to use IntelliDatum to structure their files and establish communication.

Chapter 2

Project Requirements

Our project Intelli-Datum is a web-based application that will be handling the input of single/multiple files. Once the file is uploaded the preprocessing steps begin, which includes the extraction of metadata of the file, detection of appropriate tags of the data, and at the end it will be generating the dictionary according to the single and multiple files. In our project, the actors are the client, admin, and AWS cloud processing node. Clients will be able to manage accounts/documents, upload and preprocess files, and communicate with the system. Admin will be able to manage accounts and documents. The entire service will be deployed on the AWS cloud. The following are the use cases of our project:

2.1 Use-case/Event Response Table

2.1.1 Use-case Diagram and use-case package diagram

2.1.1.1 Use-case Diagram

Following is the use case diagram of IntelliDatum.

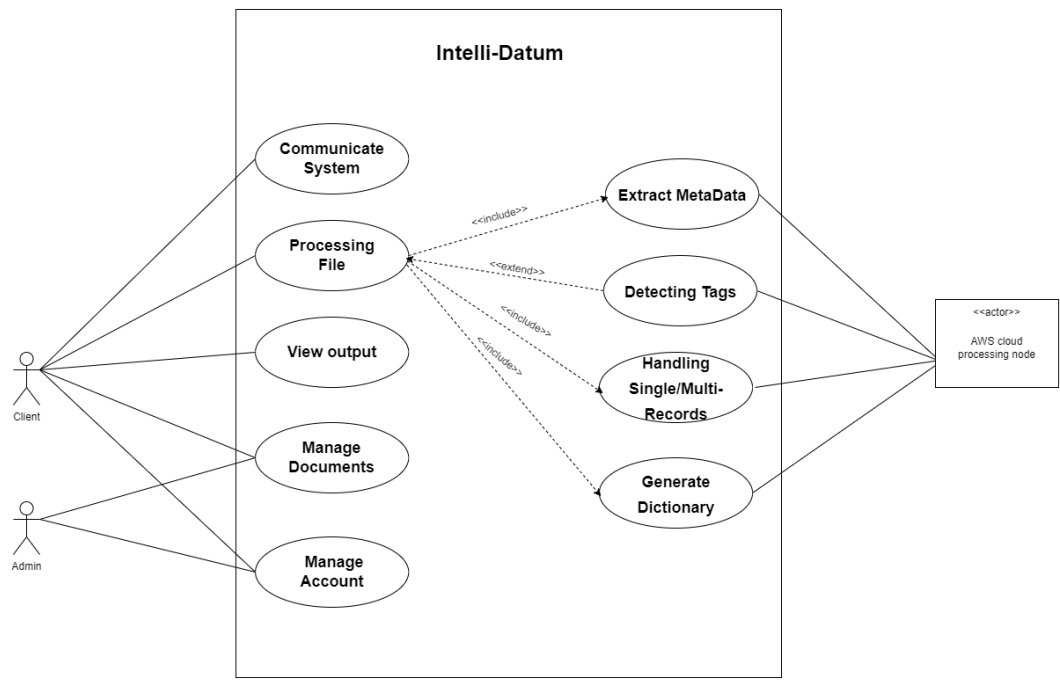


Figure 2.1: Use-case Diagram

2.1.1.2 Use-case Package Diagram

Following is the use case package diagram of IntelliDatum.

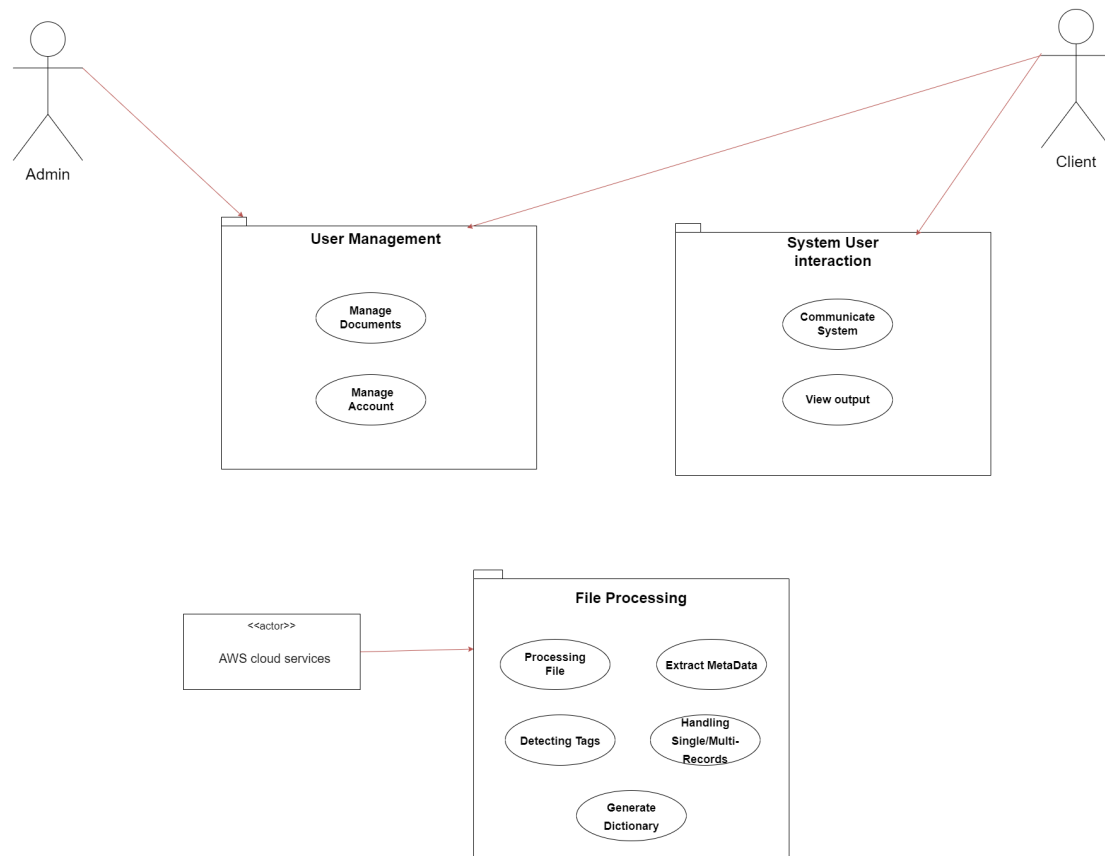


Figure 2.2: Use-case Package Diagram

2.1.1.3 Detailed Use-cases

Table 2.1: Manage Account Use Case

Use-case ID	1
Use-case name	Manage Account
Brief description	Allows users to create, update, view, or delete accounts. Admins can manage user accounts for client-related queries.
Actor(s) involved	Client, Admin
Object(s) involved	User, Account
Basic flow of events	1. User logs in by providing credentials. 2. User navigates to profile and selects settings. 3. User views information in the dashboard. 4. Information can be modified and changes are confirmed. 5. System updates the AWS database accordingly. 6. Changes are reflected in the account. 7. Admin can view and manage client accounts if needed.
Alternate flow(s)	1. If invalid data entry occurs, a message is displayed. 2. If account deletion fails, the user is notified.
Pre-conditions	Users must be logged in.
Post-conditions	Client account information is updated or deleted as requested.
Special requirements	None

Table 2.2: Manage Documents Use Case

Use-case ID	2
Use-case name	Manage Documents
Brief description	This will allow the user to manage documents such that they can upload, delete, and view the file/files that are uploaded.
Actor(s) involved	Client, Admin
Object(s) involved	User, Document
Basic flow of events	1. The user is logged in to the system. 2. The user navigates to the dashboard section of the website. 3. The user selects options to delete or upload a new file. 4. Given action is completed, changes are made in the user's dashboard. 5. Admin can manage the documents if needed.
Alternate flow(s)	If the file is not uploaded/deleted successfully, a notification is displayed.
Pre-conditions	Users must be logged into the system.
Post-conditions	The desired document is uploaded, deleted, or viewed successfully.
Special requirements	The system should handle multiple file formats.

Table 2.4: View Output Use Case

Use-case ID	4
Use-case name	View Output
Brief description	This feature allows the user to view the results of the previous use case, i.e., the generated dictionary.
Actor(s) involved	Client, AWS processing node
Object(s) involved	Files, Dictionary
Basic flow of events	1. Once logged in, the client navigates to the user dashboard. 2. The client selects the specific files view option. 3. The system proceeds to the view file page. 4. The generated output is displayed.
Alternate flow(s)	If output is not generated, an appropriate message will be displayed.
Pre-conditions	The client must be logged in and have preprocessed files to view.
Post-conditions	The client has successfully viewed the generated output.
Special requirements	The result should be in a user-friendly manner.

Table 2.5: Communicate System Use Case

Use-case ID	5
Use-case name	Communicate System
Brief description	This use case describes how to develop communication between users and Intelli-Datum, where the extracted dictionary is mapped on predefined templates to generate documents for users.
Actor(s) involved	Client
Object(s) involved	Files, Template, Dictionary
Basic flow of events	1. The client logs in to the system. 2. Client navigates to the dashboard, selects the communication option. 3. The system redirects the client to the insight generation page. 4. The client gives a prompt regarding data mapping. 5. The Template class interprets the message. 6. Data is mapped in the template according to the predefined format. 7. Clients can create a custom template. 8. The generated document can be viewed and downloaded.
Alternate flow(s)	If there is an error in mapping the fields, an error message is displayed.
Pre-conditions	Clients must be logged in and have processed files.
Post-conditions	Clients can get their data mapped in desired templates, generating documents accordingly.
Special requirements	Ensure timely responses to clients' data mapping requests.

Table 2.3: Processing Files Use Case

Use-case ID	3
Use-case name	Processing Files
Brief description	This use case handles the preprocessing tasks of the files. In the preprocessing, metadata is extracted, tags are generated, and dictionaries are created.
Actor(s) involved	Client, AWS processing node
Object(s) involved	Files, Dictionary, Records
Basic flow of events	1. Clients upload the file. 2. Intelli-Datum identifies the file type. 3. Metadata is extracted to know if it's a single record or multiple. 4. Appropriate tags are generated. 5. Based on these tags, a dictionary is generated. 6. The system returns the generated dictionary. 7. The client can view its results on the screen.
Alternate flow(s)	If the file is not supported or any failure occurs, an appropriate error will be displayed.
Pre-conditions	The client must be logged in and have uploaded the file.
Post-conditions	The file is preprocessed, and the dictionary is visible to the client.
Special requirements	Usage of AI pattern recognition for tag detection in some cases.

2.1.2 Event-Response Table

2.1.2.1 Manage account

Event	System state	System response
User logs in	The user is not authenticated and matches the username and password. The system shows the login page.	Grants access to the account to the user.
User clicks on account settings	The user is on the dashboard page	Moves to account settings options.
User views the account details	Account information is fetched from the database. The user is on the dashboard page	Account details are displayed on the screen.
User updates account information	Current account details are displayed on the screen.	Changes are confirmed and updates are reflected in the system.
Client deletes the account	The selected account already exists.	The account is deleted, and a confirmation message is displayed.
Admin accesses the dashboard	Admin has permissions and has selected settings.	Displays a list of all the client accounts.

2.1.2.2 Manage documents

Event	System state	System response
User logs in to the system	The user is on the login page	The user is authenticated and logs in to the account.
Client clicks on upload	The client is on the dashboard page	Allows the client to upload a file into the system.
Client views already uploaded documents	The client is on the dashboard page	A list of already existing files is displayed.
Client selects the file	The client is on the dashboard page	The file is selected successfully.
Client clicks on the delete option	The document exists and is listed in the dashboard's file section.	The document is deleted, and a confirmation message is displayed.

2.1.2.3 Processing files

Event	System state	System response
Client uploads a file	The client is on the dashboard page	The client successfully uploaded the file.
System identifies the file type	The file is uploaded, and the user is on the dashboard.	The status bar shows the progress.
System extracts metadata	File processing has started.	Metadata is extracted to determine whether it is a single or multi-record file.
System generates tags	Metadata extraction is complete.	Appropriate tags are generated, and the status bar moves forward.
System generates dictionary	Tag generation is complete.	A dictionary is generated and results are stored. The status bar turns green, indicating successful completion.

2.1.2.4 View output

Event	System state	System response
Client logs in to the system	The client is on the login page.	The client is authenticated.
Client selects view option from the dashboard	The client is on the dashboard page.	Navigates to the view page and views the generated output.
Client selects download option from the dashboard	The client is on the dashboard page.	The output file is downloaded to the local system for further inspection.

2.1.2.5 Communicate system

Event	System state	System response
Client logs in	The user is on the login page.	The user is authenticated and navigated to the dashboard.
Client selects a file	The client is on the dashboard section.	The file is selected, and the options are displayed.
Client selects communication option	The client is on the dashboard page.	Navigates to the communication page.
Client types a prompt	The client is on the communication page, and the file is processed.	The prompt is successfully sent to Intelli-Datum, which creates templates according to user specifications. The data fields are mapped, and the desired document is displayed.
Client views the document	The client is on the communication page, and communication has been initiated.	The final document is displayed and can be downloaded.
User selects clear option	The client is on the communication page with messages exchanged between the client and Intelli-Datum.	The messages are cleared.

2.2 Functional Requirements

We have listed the functional requirements of Intelli-Datum according to the modules that are listed below:

2.2.1 File Input Module

- The system shall allow the client to upload the file in multiple formats like CSV, XML, fixed length, and unstructured line data.
- The system shall provide a user-friendly interface for file uploading.
- The system shall display the progress of the file that is being uploaded.
- The system shall allow the user to upload at least 10 multiple files.
- The system shall validate the size and format of the file that's being uploaded to ensure that there is no compatibility issue.

2.2.2 Delimiter Separated File Handler Module

- The system shall correctly identify the metadata of the file to know if it's a single or multiple record.
- The system shall correctly identify the delimiter placed in the file, supporting multiple delimiters like ',' and '|'.
- The system shall handle the files with or without headers effectively.
- The system shall generate the tags of data by giving them user-friendly names.
- The system should handle single/multiple records and documents effectively and generate dictionaries accordingly.

2.2.3 Fixed Length File Handler Module

- The system shall extract the metadata of the file accurately.
- The system shall process single and multiple records/documents effectively.
- The system shall process the fixed length based on inferred field lengths.
- The system shall name the identified fields with user-friendly names.

2.2.4 XML File Parser Module

- The system shall process XML files such that the tags and attributes are used for effective tag generation.
- The system shall give user-friendly names to the tags that are generated.
- The system shall support XML schema validation.

2.2.5 Line Data Handler Module

- The system shall apply pattern recognition techniques to detect the context and location of fields and infer relationships between them.
- The system shall handle line data files without obvious structure.
- The system shall use heuristic methods for field mapping.
- The system shall give user-friendly names to the fields that are detected.

2.2.6 WebApp

- The system shall provide a user-friendly interface for all the functionalities that are mentioned.
- The system shall allow the user to easily navigate between different pages of the website.
- The system shall allow its users to submit feedback for the actions they have performed.

2.2.7 Data Mapping

- The system shall allow the user to input the message related to data that is needed to be interpreted and mapped on the template.
- The system shall support the modification of the template.
- The system shall allow the user to map the extracted data fields to the specific placeholders in the template.
- The system shall automatically generate documents based on mapped data and selected templates.
- The system shall allow the user to view the generated document before downloading.

2.3 Non-Functional Requirements

The following are the non-functional requirements of the system:

2.3.1 Reliability

The system failure occurs when the software is unable to perform according to the desired working. The consequences of the system loss can result in information loss and incorrect outputs. In case of failure, the system must roll back to the previous step and restore the activity in less than 5 minutes. To maintain reliability, the system must implement backup logs for the operations. When the system is processing the file and a certain error occurs, we will be generating a clear and detailed message and will alert for any failures.

2.3.2 Usability

The system's interface will be user-friendly with clear instructions and navigation, allowing new users to work on this easily without a tutorial. Our application will allow the user to view the previous documents that are generated using a single interaction. In case of failure, the system provides clear and actionable steps in detail. The system will offer a responsive design so that it will work on various device types.

2.3.3 Performance

The system should be able to upload a file within 5 seconds for a size less than 10MB. Users can also upload files simultaneously, that is around 10 files in one go. The system should interpret the user's prompt within 10 seconds. Our system will be mapping the data to templates within 5 seconds.

2.3.4 Maintainability

For the backend services, we will be opting for CI/CD pipelines so that code will be maintained easily and it will be much easier for us to update the code.

2.4 Domain Model

The domain Model diagram of our system consists of 3 main hot-spots. The user-related entities, the file-related entities and the dictionary-related entities. Following is the domain model of our system, highlighting the main concepts of our requirements:

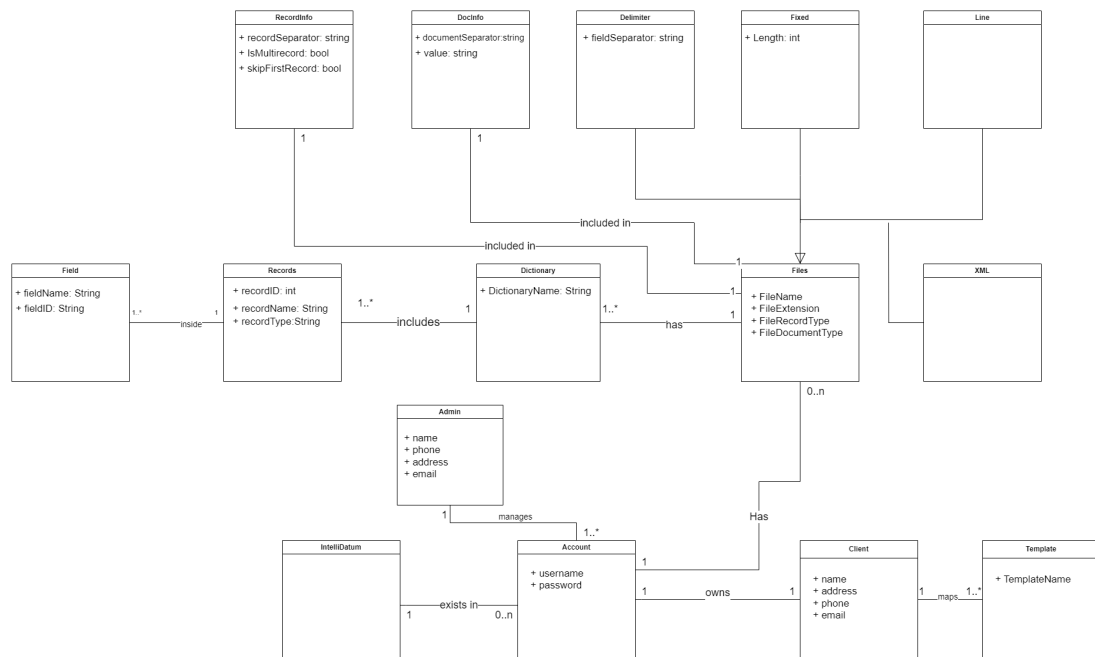


Figure 2.3: Domain Model Diagram

Chapter 3

System Overview

The system has three main components to provide functionality. The first is the website interface through which the user with access interacts with the system. The second is the processing node which will be deployed as a service on AWS and is connected to the website through an API. Lastly the Account management system helps in the management and interaction with account data and files, providing tools for users and administrators.

3.1 Architectural Design

3.1.1 Client

The client subsystem features a web application through which users can submit their files. These files are uploaded and verified if the file type is valid. Then an API POST request is sent to the processing node.

3.1.2 Cloud Service

In the cloud service, the major processing is carried out. Once the file is received, the file's metadata is extracted and the appropriate content is processed based on the file type and record type. The data tagging module is then applied if necessary. Lastly, using the extracted data, the Dictionary is generated. This extracted dictionary is sent back to the client in the appropriate file format, such as an .epf (Elixir Product File).

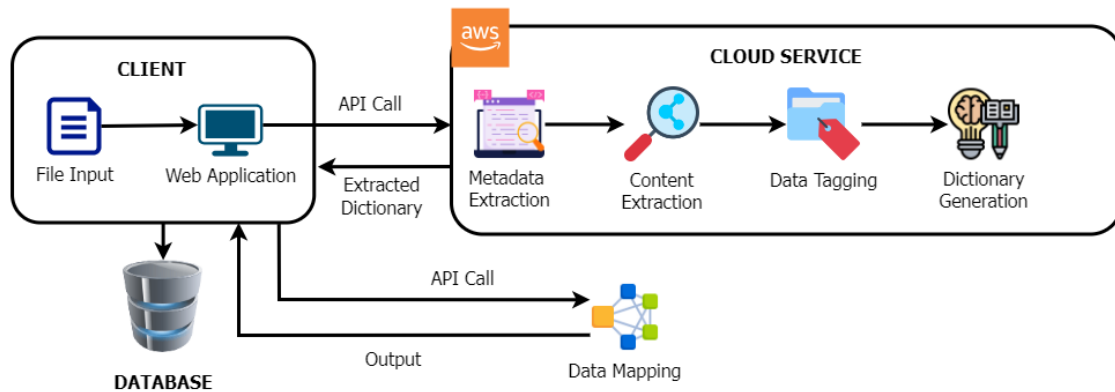


Figure 3.1: Architecture Diagram

3.1.3 Database

Most of our system focuses on processing. The only time the database is required is to save user account information and recently uploaded files.

3.1.4 Data Mapping

The client files are then mapped onto the dictionary generated by the processing node. This mapping is then reflected on a template, which is used to send email messages to recipients. This marks the final output.

width=1/classDiagram.drawio.png

3.2 Design Models

3.2.1 Activity Diagram

The system has four swimlanes representing the different subsystems. The main processing is performed in the *Processing File* subsystem.

3.2.1.1 Client Interface

The client interface connects the system with other subsystems. It is crucial for user experience.

3.2.1.2 Processing File

When the client submits the prompt and the file to the system, the file is validated and checked for multiple or single records. Based on its file type, the following modules are executed:

- Delimiter Detection
- Tag Generation
- Length Generation
- File Parsing
- Pattern Recognition
- Heuristic-Based Field Mapping

3.2.1.3 Communication Processing

If a prompt is provided with the file, an NLP model analyzes the prompt and uses the dictionary file to map the data onto the template. This information is used in the *Processing File* subsystem.

3.2.1.4 Document Management

Whenever a file is uploaded, it is automatically cached in the system as *recent files*, which the user can access for 48 hours. After that, it is deleted from the system.

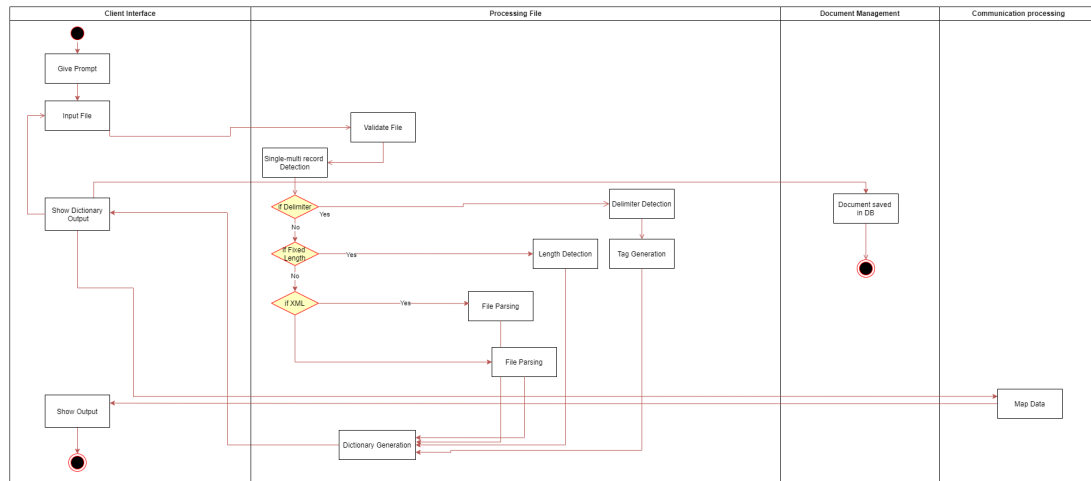


Figure 3.2: Activity Diagram

3.2.2 Class Diagram

3.2.2.1 User Classes

The user classes describe the relationship between classes related to user management and file management. The user relation classes include:

User: The *User* class is the parent class of *Admin* and *Client* classes. Every user also has an account.

Client: The *Client* class describes the functionality of a client user. It can upload files and process files. Each client has files.

Admin: The *Admin* class has the authority to manipulate user accounts and files.

Account: Every user has an account through which they can login, logout, and manage the account.

3.2.2.2 File Classes

The file relation classes include:

File: The *File* class is the heart of the diagram. It contains file information such as file name, file type, document type, etc. Through this class, the system calls various methods to help in file processing and dictionary generation. The *File* class is the parent of four main file types: *Delimited*, *Fixed Length*, *XML*, and *Line*.

Delimited: The *Delimited* class is a child of the *File* class. It is differentiated by the field separator attribute.

Fixed Length: This class represents the *Fixed Length* file type, which has a fixed length of characters for each field and record.

XML: This class represents the *XML* file type.

Line: This class represents the *Line* file type, which requires pattern recognition of fields and records.

Template: The system can generate communication mediums to send to customers by mapping the dictionaries to files onto a template.

3.2.2.3 Dictionary Classes

The dictionary relation classes include:

Dictionary: The *Dictionary* class helps with dictionary creation and gives structure to the output.

RecordInfo: The *RecordInfo* class provides header information in the dictionary, indicating record separators in multi-line and single-line files.

DocInfo: This class gives document information, such as how the document is separated.

Records: Each *Dictionary* contains a record. The *Records* class contains information such as *recordID*, *recordName*, and *recordType*.

Field: The *Records* class contains fields. These *Fields* hold information about the fields in the file.

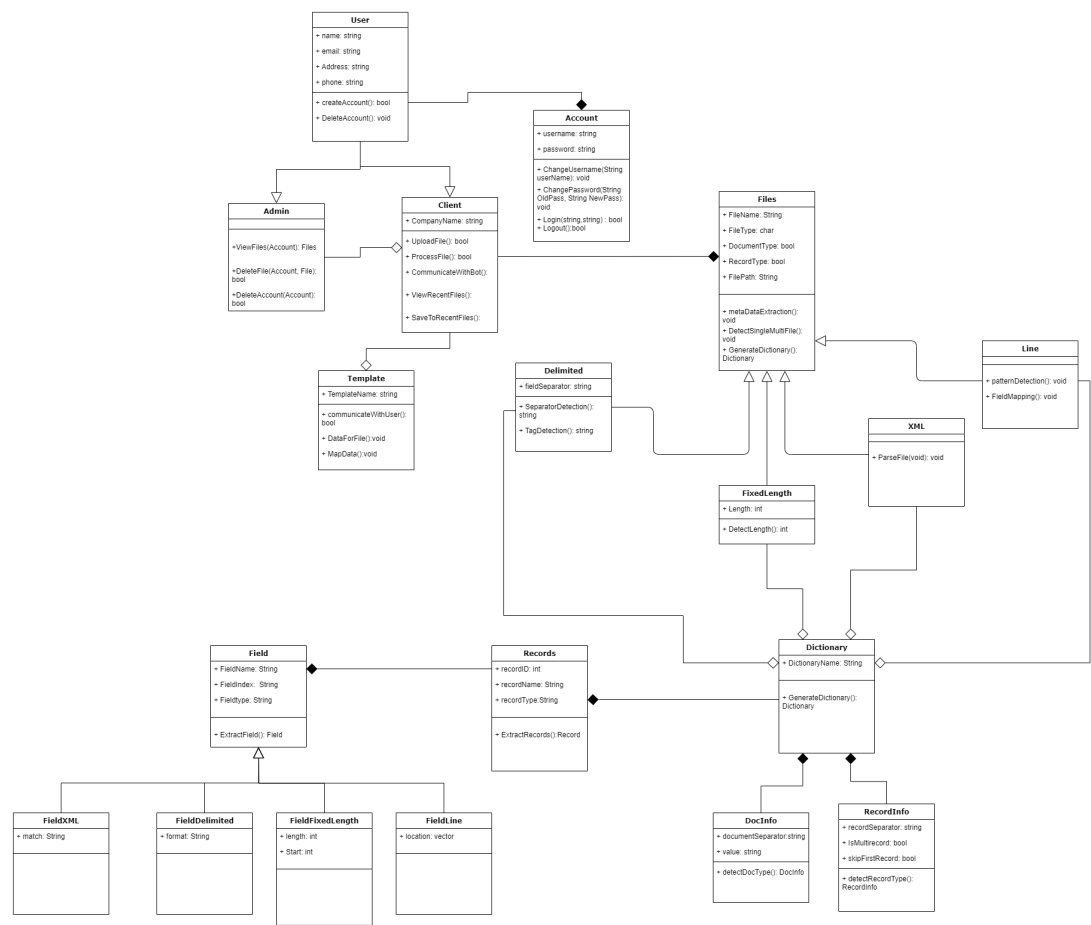


Figure 3.3: Class Diagram

3.2.3 System Sequence Diagram

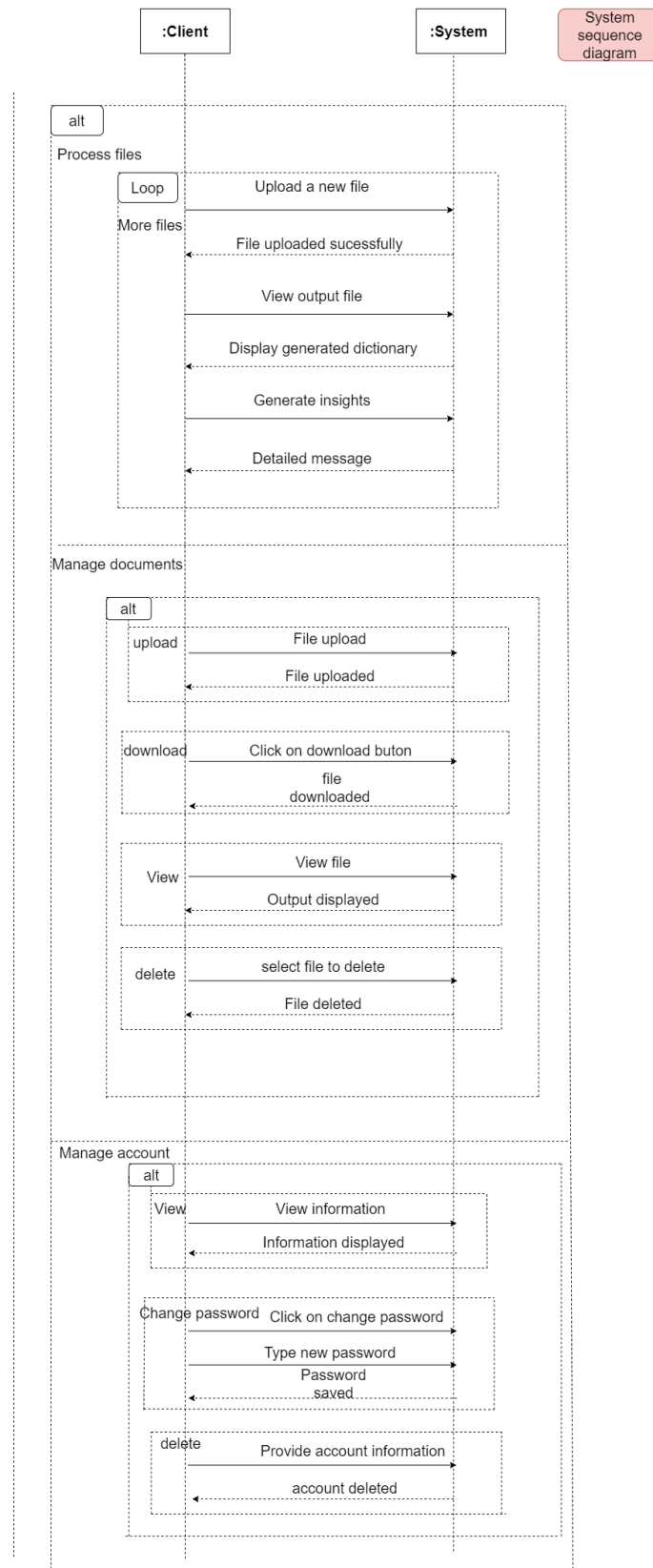


Figure 3.4: System Sequence Diagram

3.2.4 Sequence Diagram

The sequence diagrams show the backend operations that are extensively carried out in this system.

3.2.4.1 Processing File

The file is processed using different models and techniques for each separate file type. However, the standard protocol for this is reflected in the sequence diagram below. We have a dictionary class that calls its corresponding sub-component functions to process the file.

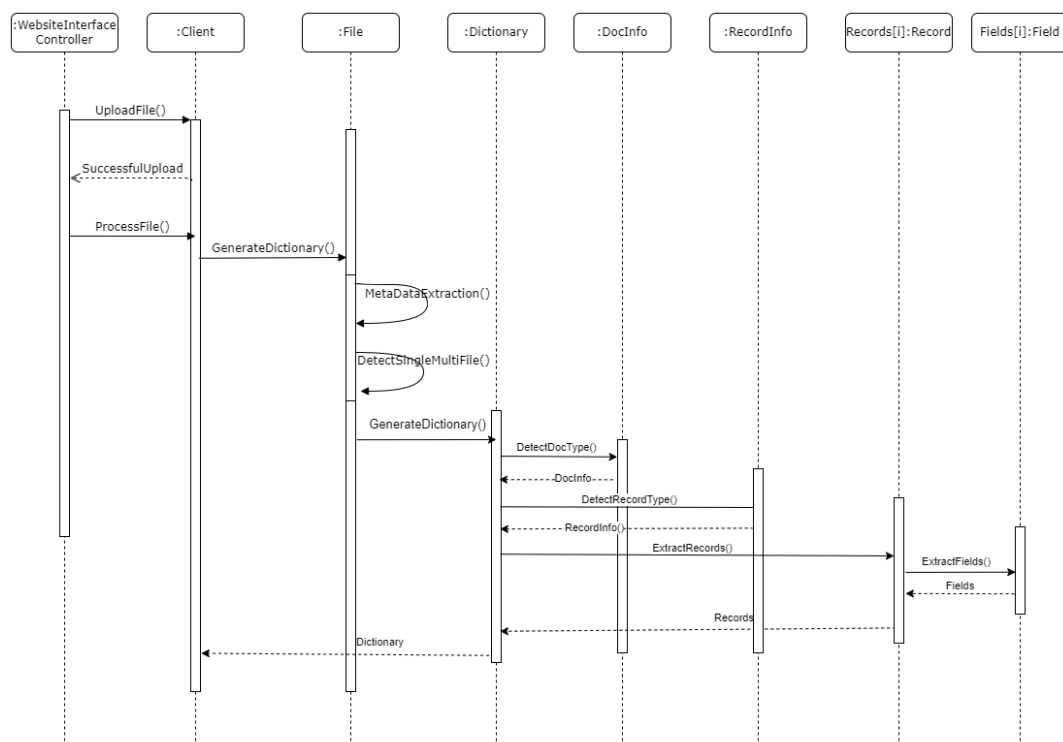


Figure 3.5: File Processing Sequence Diagram

3.2.4.2 Manage Accounts

The manage account sequence diagram shows how a user can perform various operations. The manage account functions highlighted in this diagram are:

- CreateAccount

- ChangeEmail
- ChangePassword
- DeleteAccount
- ViewInformation

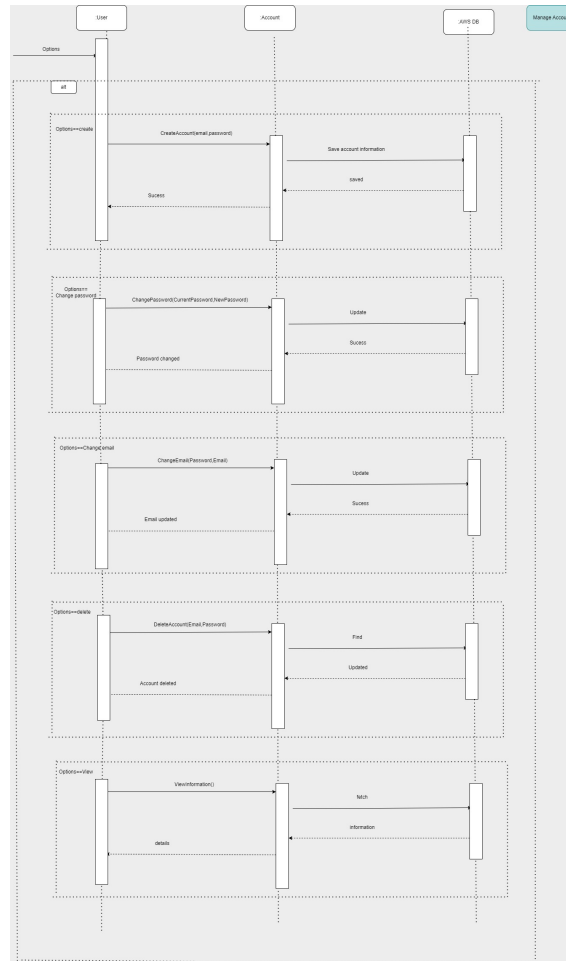


Figure 3.6: Manage Account Sequence Diagram

3.2.4.3 View Output

To view the output, it is necessary to first generate a dictionary of a file and then map the dictionary. Once these processes are performed, the system automatically saves the file object to "recently saved."

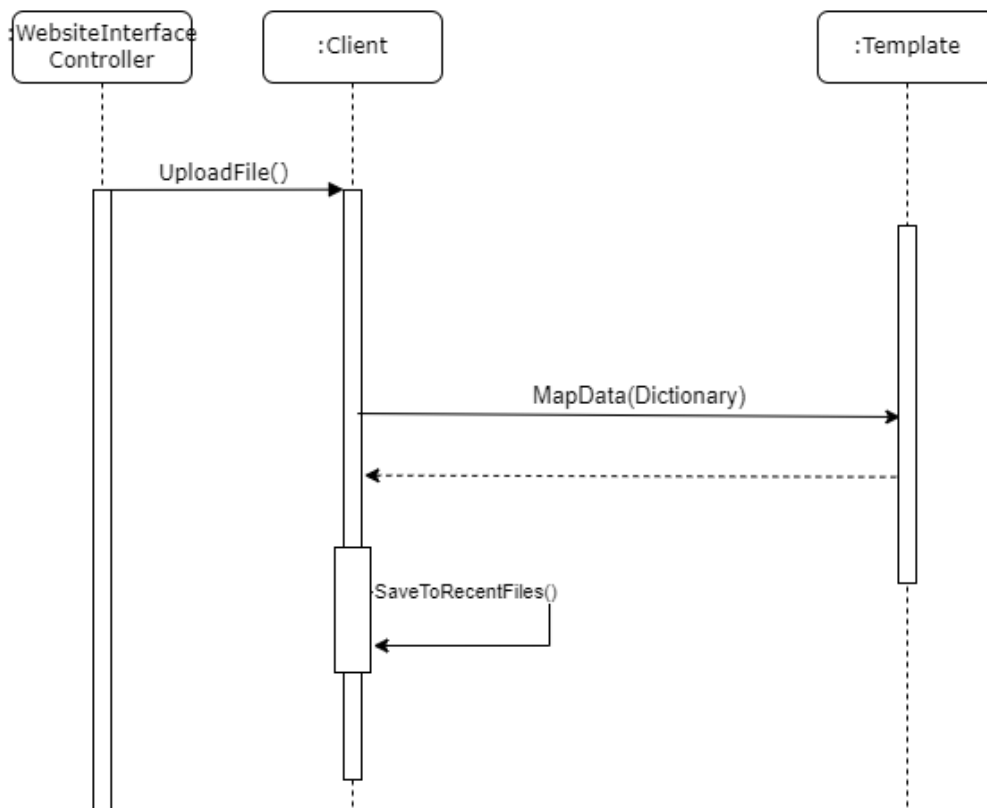


Figure 3.7: Output View Sequence diagram

3.2.4.4 Manage Documents

Viewing and managing documents is not the primary focus of the project, but it is important for user experience. This part of the system handles the functionalities of saving and viewing files. When this use case is reflected in the admin role, it provides the privileges of deleting or viewing files.

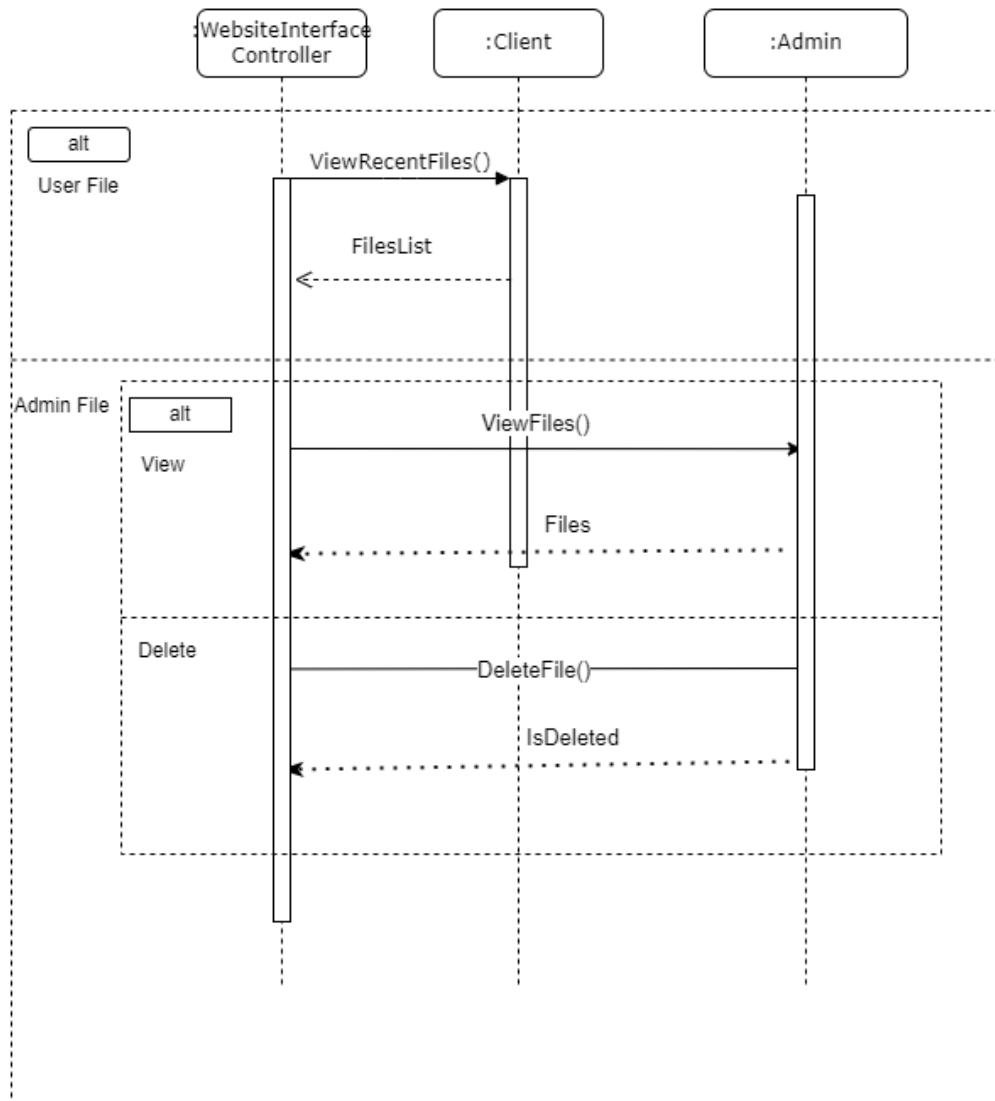


Figure 3.8: Manage Documents Sequence Diagrams

3.2.4.5 Communicate with User

This part of the system uses the generated dictionary to map the data onto a template. This template is used to create customer messages and send them out through emails. The template uses AI to map the dictionary and file data onto itself.

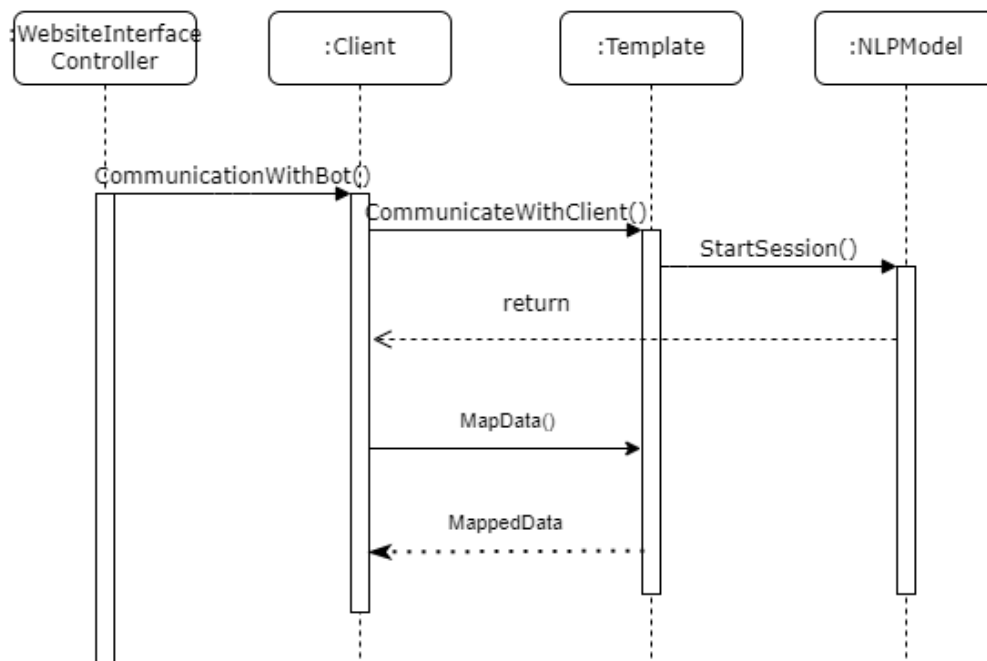


Figure 3.9: Communicate with User Sequence Diagram

3.2.5 State Transition Diagram

The state transition diagram shows how the different events in the system affect the outcome. Analyzing the diagram helps us see that, mostly, if there is any error after an event during the processing time, the system automatically redirects to the input page. Upon successful dictionary generation, the system reaches the final state.

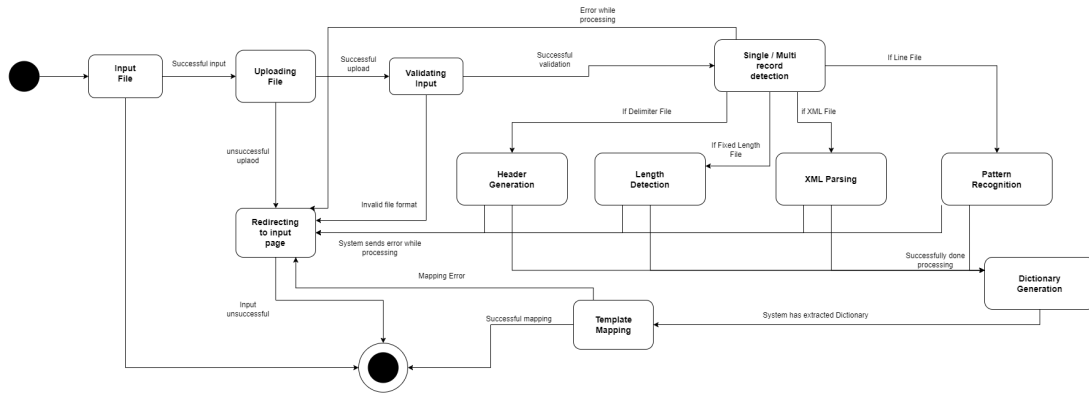


Figure 3.10: State Transition Diagram

3.2.6 Deployment Diagram

The deployment diagram for our system illustrates the physical architecture of the solution, showing how different components are deployed on the cloud and interact with one another. The system consists of multiple nodes: the client node where users interact via a web interface, the processing node hosted in the cloud, and the database server for storing user accounts and recent files. The client submits files through the web interface, which are sent to the processing node via API requests. A small database server is used to store user credentials and metadata of recently processed files, while the system's communication features operate within the processing node to map data to templates for communication purposes.

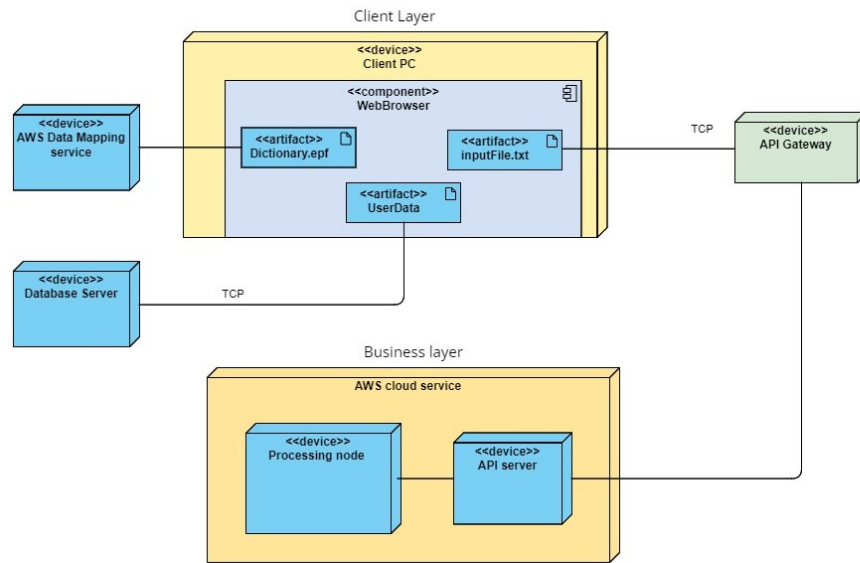


Figure 3.11: Deployment Diagram

3.2.7 Data-Flow Diagram

The data flow diagram, though not necessary for OO-Design project, helps in understanding the movement of data in our system. The client submits a file through the web interface, which is then sent to the processing node. The processing node validates the file, extracts metadata, and generates a dictionary. This dictionary is either returned to the client or used for mapping onto templates for communication. The database stores user credentials and recent files, with periodic interactions for retrieval and storage during file processing and account management.

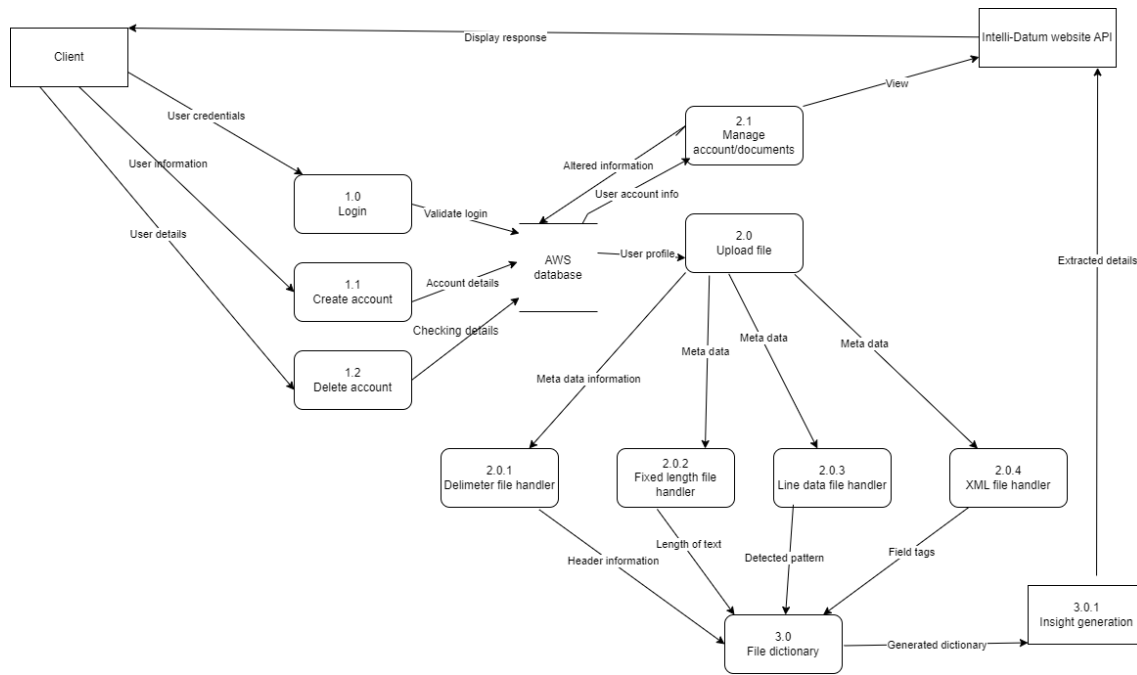


Figure 3.12: Dataflow Diagram

3.3 Data Design

IntelliDatum receives input in the form of files that are categorized into four types of files. These include:

1. **Delimiter Separated Files:** Files where fields are separated by delimiters such as commas (,), pipes (|), or other characters.
2. **Fixed Length Files:** Files where fields have a fixed length, with each field starting and ending at predefined character positions, and fields are separated by space.
3. **Line Data Files:** Files without any obvious structure, where each line may represent data points in different formats.
4. **XML Files:** Files with structured, hierarchical data where the schema is embedded in the file.

These input files are processed to create a structured data dictionary which is stored in an EPF file. The dictionary stores the metadata of the file that includes details like file type, whether it is a multi-record or single record file, whether it is a multi-document or single document file, and many more. Additionally, the dictionary also stores record names, field names, and field data types in a standard format. This structured dictionary serves as a foundation for subsequent processes like data mapping, insight generation, etc.

```

<data-map name="Orders" type="Delimited" encoding="iso-8859-1" version="1.0.1">
  <doc-info separator="CharSequence" value="\r\n">
    <record-info multi-record="False" rec-id-length="0" rec-length="0" separator="\r\n" skip-first-record="True"/>
    <field-info end-delimiter="&quot;" separator="," start-delimiter="&quot;"/>
  </doc-info>
  <data-layout>
    <record name="OrderInfo" type="Repeat">
      <field index="1" name="OrderID" type="String"/>
      <field index="2" name="CustomerID" type="String"/>
      <field index="3" name="EmployeeID" type="String"/>
      <field index="4" name="OrderDate" type="String"/>
      <field index="5" name="RequiredDate" type="String"/>
      <field index="6" name="ShippedDate" type="String"/>
      <field index="7" name="ShipVia" type="String"/>
      <field index="8" name="Freight" type="String"/>
      <field index="9" name="ShipName" type="String"/>
      <field index="10" name="ShipAddress" type="String"/>
      <field index="11" name="ShipCity" type="String"/>
      <field index="12" name="ShipRegion" type="String"/>
      <field index="13" name="ShipPostalCode" type="String"/>
      <field index="14" name="ShipCountry" type="String"/>
    </record>
  </data-layout>
</data-map>

```

Figure 3.13: Example Dictionary

3.3.1 File Processing Overview

1. **Delimiter Separated Files:** The first step is to parse the file and identify the delimiter being used. Next, it is determined if the file is a multi-document or a single-document file, and in the case of multi-document files, the document separator is also identified. Then it is determined if the file is a multi-record or a single-record file, and the number of varying records in a file is identified. Headers are identified, if the file contains any, and mapped to the first record. Finally, record and field names are generated, and a data dictionary is created.
2. **Fixed Length Files:** The first step is to parse the file and identify the record and field lengths. Next, it is determined if the file is a multi-document or a single-document file, and in the case of multi-document files, the document separator is also identified. Then it is determined if the file is a multi-record or a single-record file, and the number of varying records in a file are identified. Headers are identified, if the file contains any, and mapped to the first record. Finally, record and field names are generated, and a data dictionary is created.
3. **XML Files:** Given that XML files already have a structured format, IntelliDatum parses the file and extracts the dictionary from appropriate tags in the files.
4. **Line Data Files:** As line data files lack any proper structure, the records and fields are identified by using the data position and context in the file. The appropriate names are generated for these records and fields and stored in the form of a structured dictionary.

3.3.2 Data Mapping

IntelliDatum supports a data mapping feature where a TDF file is used to define document templates. Fields from the data dictionary are mapped to these templates, enabling the generation of documents with predefined formats using the data extracted from the files. The user can input a message, and the model interprets it to create a communication, and generates documents according to the defined template.

3.3.3 Data Storage

IntelliDatum web app uses the NoSQL Database, MongoDB, to store the user account details, the unstructured input files, the data dictionaries generated from input files, the templates used for data mapping, and the documents generated from these templates. The data relationship is displayed in an ERD (Entity Relationship Diagram) displayed below.

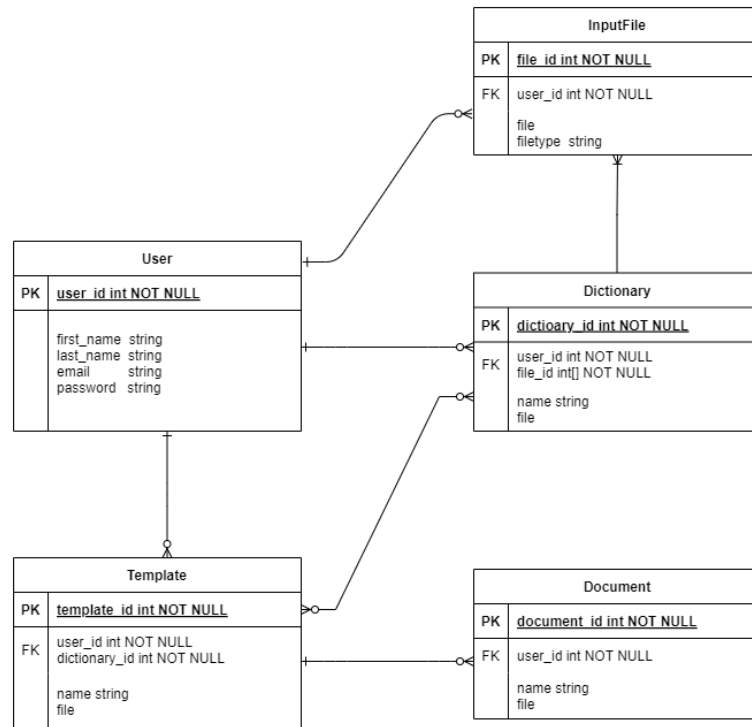


Figure 3.14: Entity Relationship Diagram

The provided data design for IntelliDatum is used to achieve its goal of automating the process of transforming unstructured data files into structured dictionaries and integrating them with predefined templates, allowing the users to generate documents with ease. This data design ensures IntelliDatum is a reliable and scalable solution that meets the needs of enterprise users handling diverse and complex data.

Chapter 4

Conclusion and Future Work

In conclusion IntelliDatum is a major advancement in the field of data management. It converts unstructured files into human understandable structured dictionaries with the help of Machine Learning and Generative AI. IntelliDatum automates the initial steps of data management and lays the foundation for further use of the data in the form of data mapping, data analysis, insights generation, and so much more. The collaboration with Elixir Technologies helps understand the problem in more detail as the company is currently facing the issues due to manual data management and dictionary extraction. IntelliDatum would help them resolve the issues and provide an efficient and time saving solution that fulfills their real world needs.

IntelliDatum is a web application that handles the processing of various types of files creating a standard dictionary and mapping data to templates to create meaningful documents from the data files. The document lists down the major features of IntelliDatum and its functional and nonfunctional requirements.

All in all , IntelliDatum brings a reliable and efficient solution to the world of data management, making use of artificial intelligence to give meaning to the complex data files and making them usable in this fast paced tech world. IntelliDatum lays a good ground for successive innovations in the field of data management for example including more data file formats and the adoption of superior artificial intelligence. The current development puts IntelliDatum as a tool of high worth in the realm of smart-data management.

Bibliography

- [1] J. Baig. Unstructured data challenges for 2023 and their solutions, December 22 2022. Retrieved from Astera website: <https://www.astera.com/type/blog/unstructured-data-challenges>.
- [2] MongoDB. Unstructured data: What it is and why it matters. <https://www.mongodb.com/resources/basics/unstructured-data>. Accessed: 2024-09-12.
- [3] Tableau. The importance of data management for data-driven decision making, 2023. Retrieved from Tableau Software website: <https://www.tableau.com/learn/articles/what-is-data-management>.